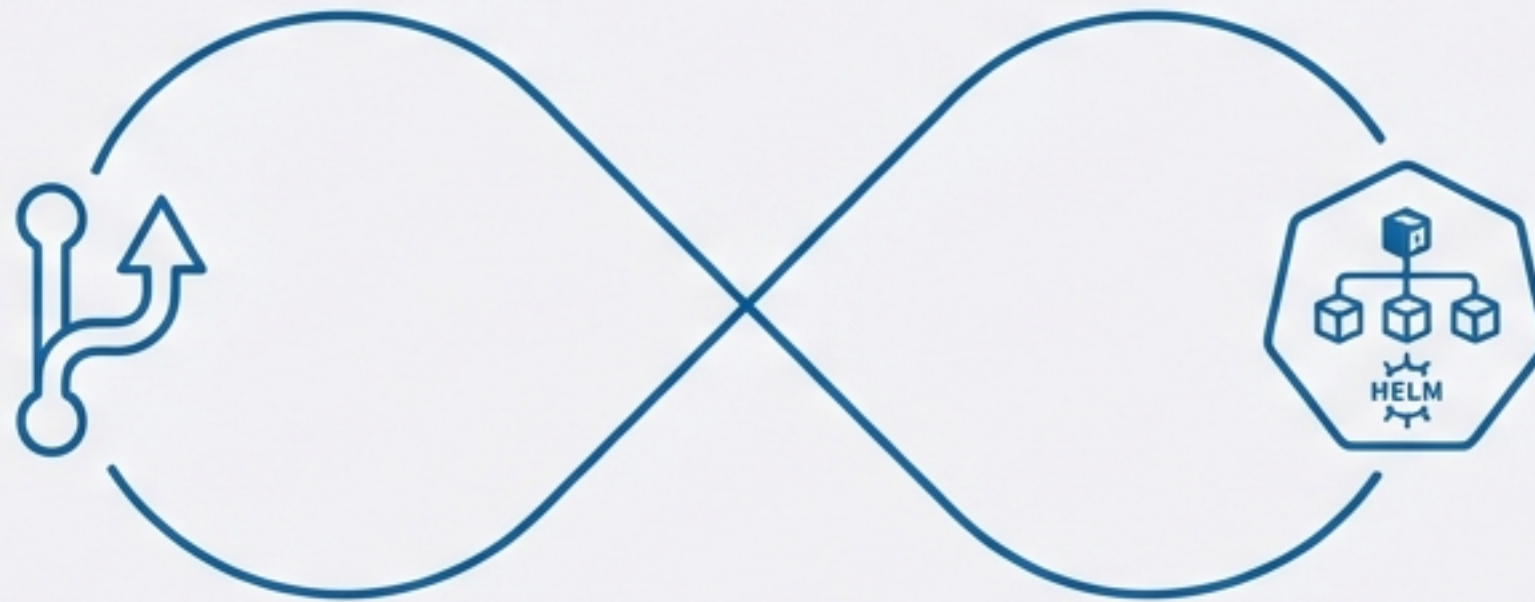


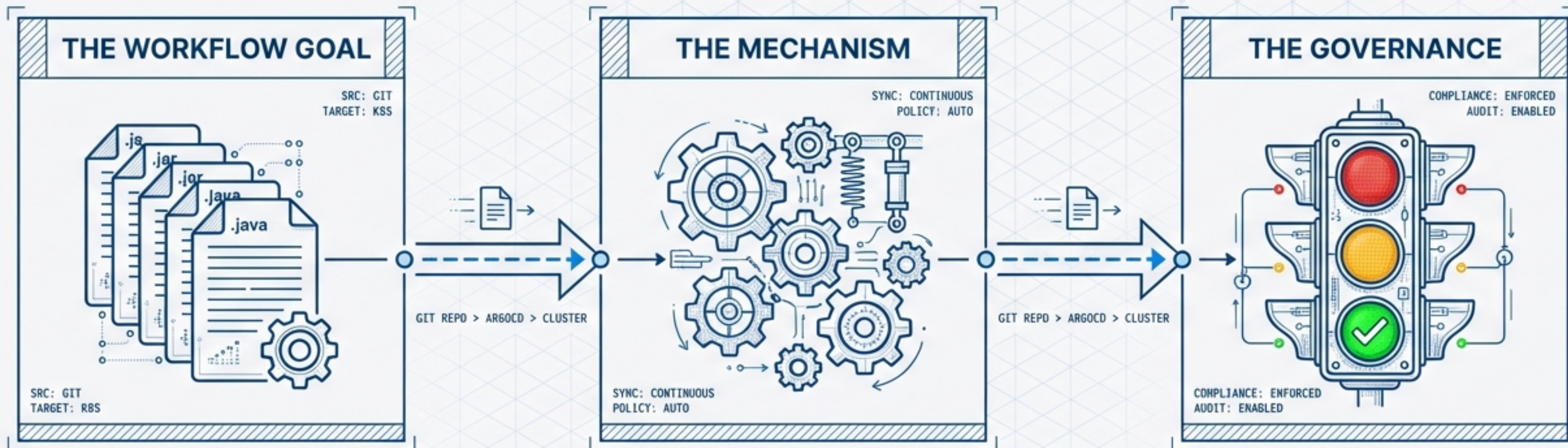
Advanced ArgoCD Operations & CI/CD Workflows

Mastering Automation Policies, Lifecycle Management, and Governance



SCOPE: ReactJS Auto-Deploy • Spring Boot Services • Self-Healing • Pruning • Sync Windows

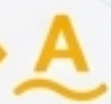
Operational Roadmap



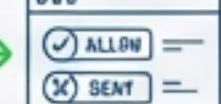
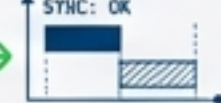
Deploying Application Code

- ReactJS Auto-Deploy
- Spring Boot File Upload Service

ArgoCD Behavioral Rules

- Self-Healing (Drift Correction)  → 
- Self-Prune (Garbage Collection)  → 

Lifecycle Control

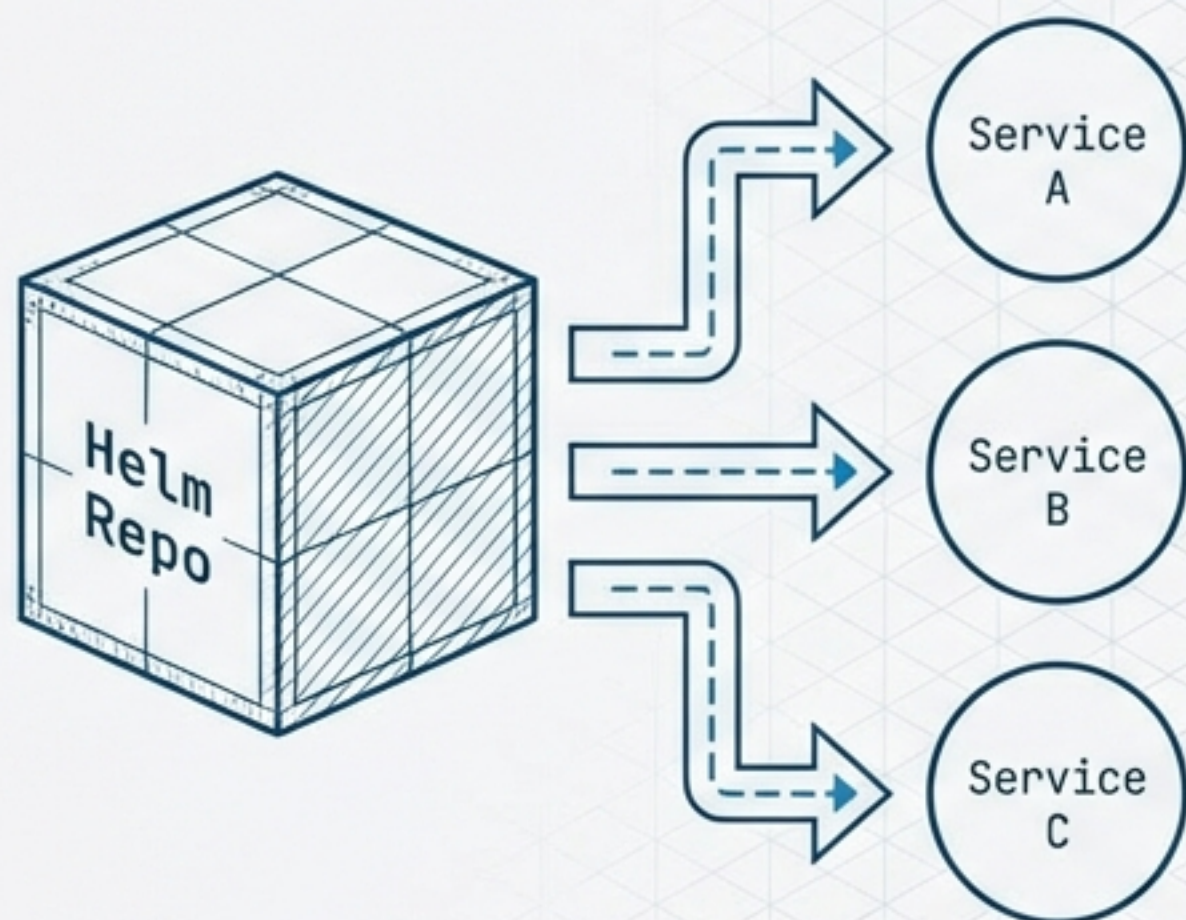
- Propagation Policies  → 
- Sync Windows (Time-based Rules)  → 

Helm Repository Strategy

Architectural Prerequisite for Service Deployment

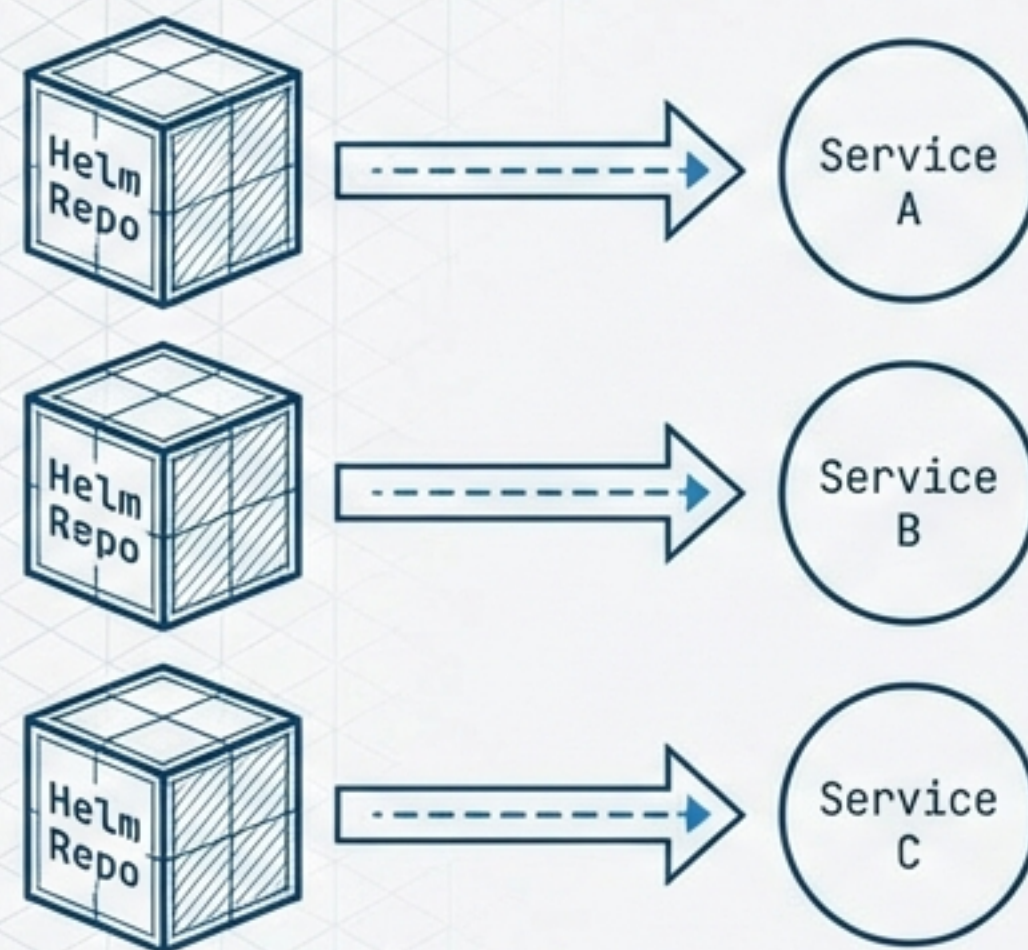


Option A: Monolithic Repo



One Helm-Repo for all service definitions.
Centralized management.

Option B: Distributed Repos



One Helm-Repo per service.
Decoupled lifecycles and isolation.



The Philosophy: Git is the Source of Truth

Git
State



Cluster
State



DRIFT DETECTED

When Reality Diverges from Definition:

- Manual "kubectl edit"
- Hotfixes applied directly to cluster
- UI modifications bypassing Git

Enforcing Consistency: Self-Healing

1. Detect Drift

Identifies manual changes or inconsistencies.



2. Reapply State

Automatically pulls definition from Git.



3. Enforce Specs

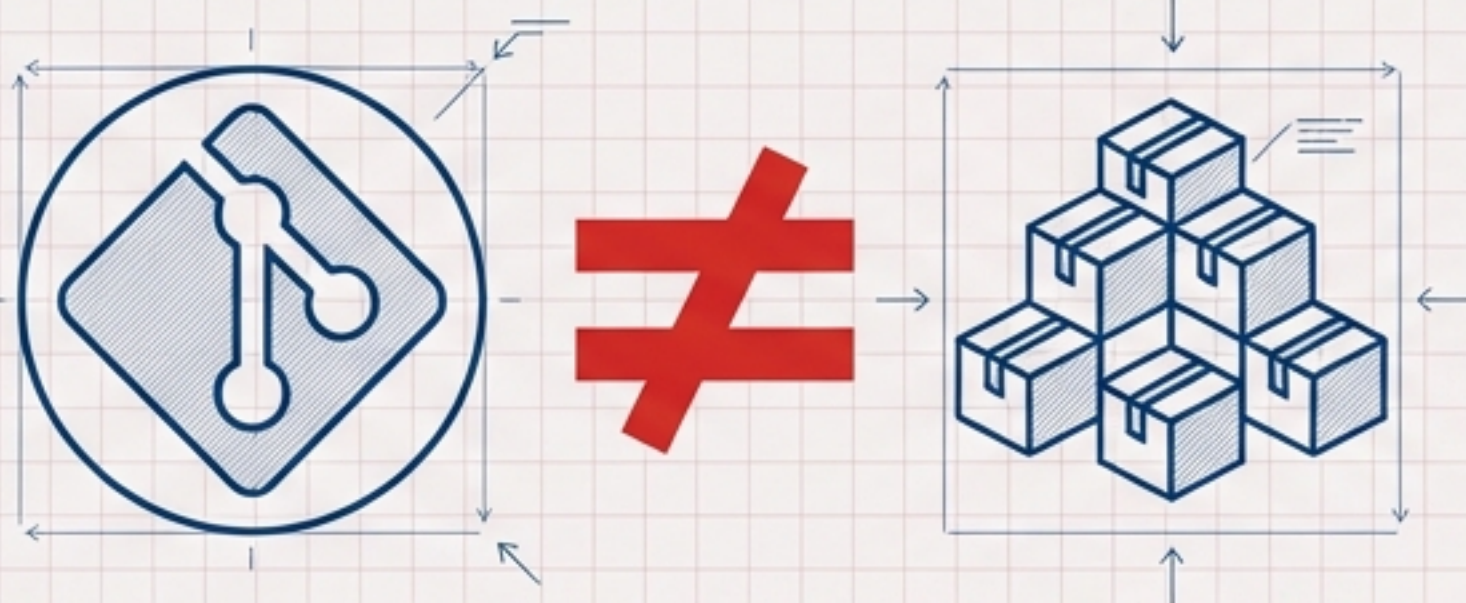
Scales/Configures back to Git baseline.



Result: System acts as an auto-correction mechanism against "kubectl edit" or UI changes.

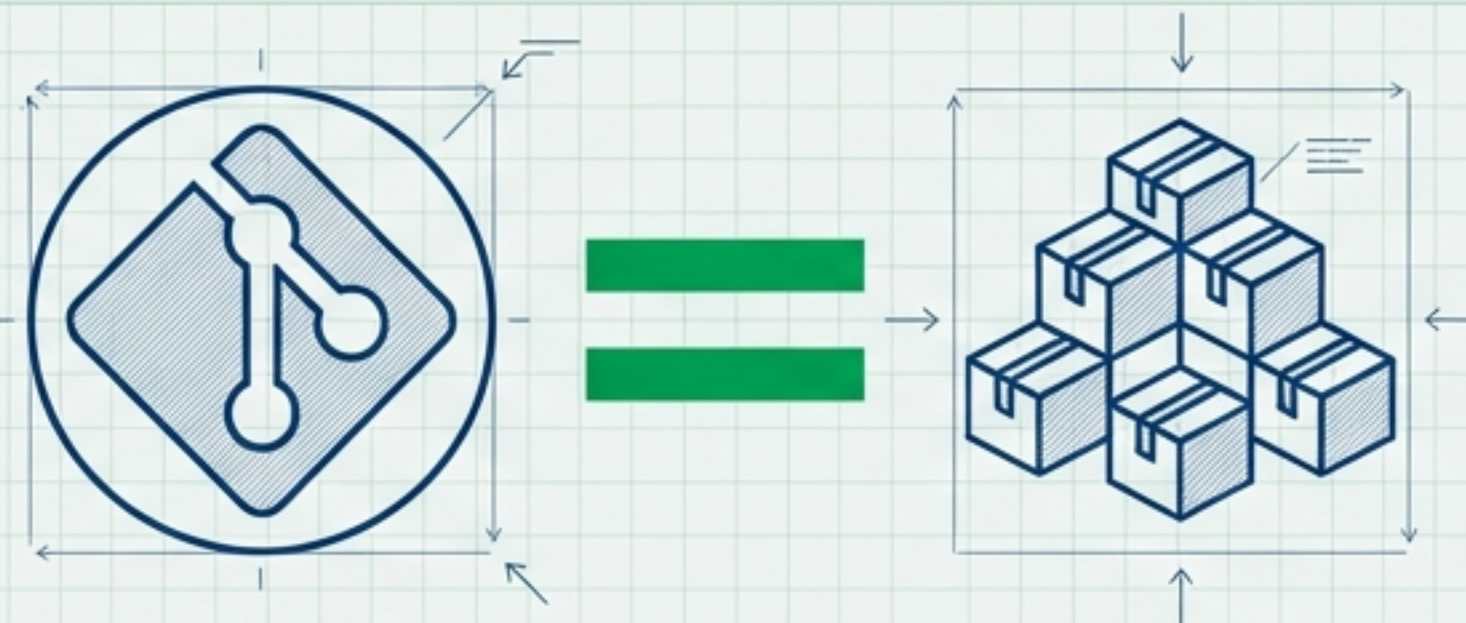
The Impact of Self-Healing

Without Self-Healing



- Cluster **≠** Git (Inconsistency)
- Status: **OutOfSync**
- Action: **Manual intervention** required

With Self-Healing

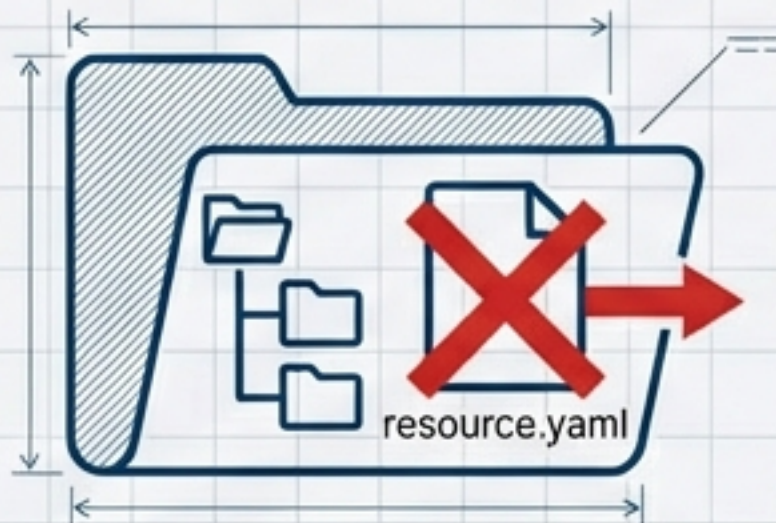


- **Drift** is immediately **countered**
- **Automatic re-application** of Git state
- Action: **System auto-corrects** to defined replicas

Resource Hygiene: Self-Prune

Handling Deleted Resources

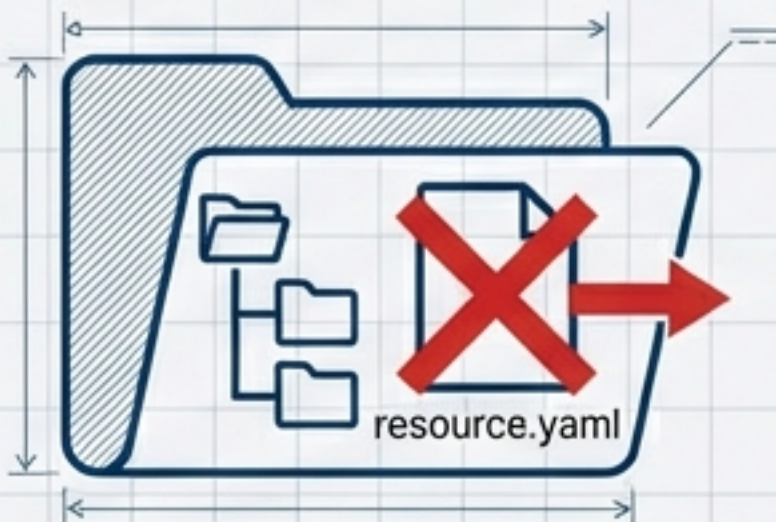
Scenario A: Without Prune



Ghost Resource

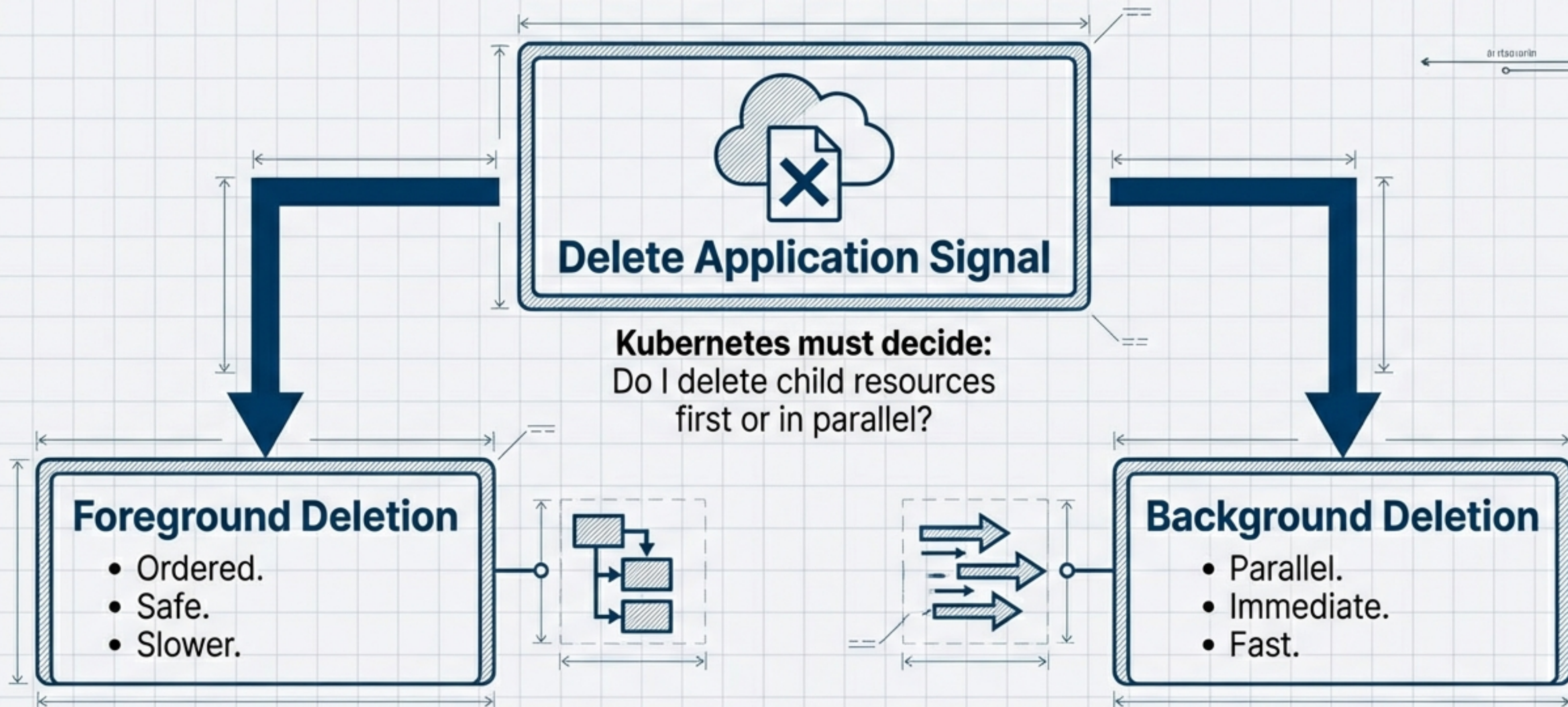
Resources keep running forever, consuming quota even after removal from Git.

Scenario B: With Prune



Resource is deleted from the cluster the moment it is removed from Git.

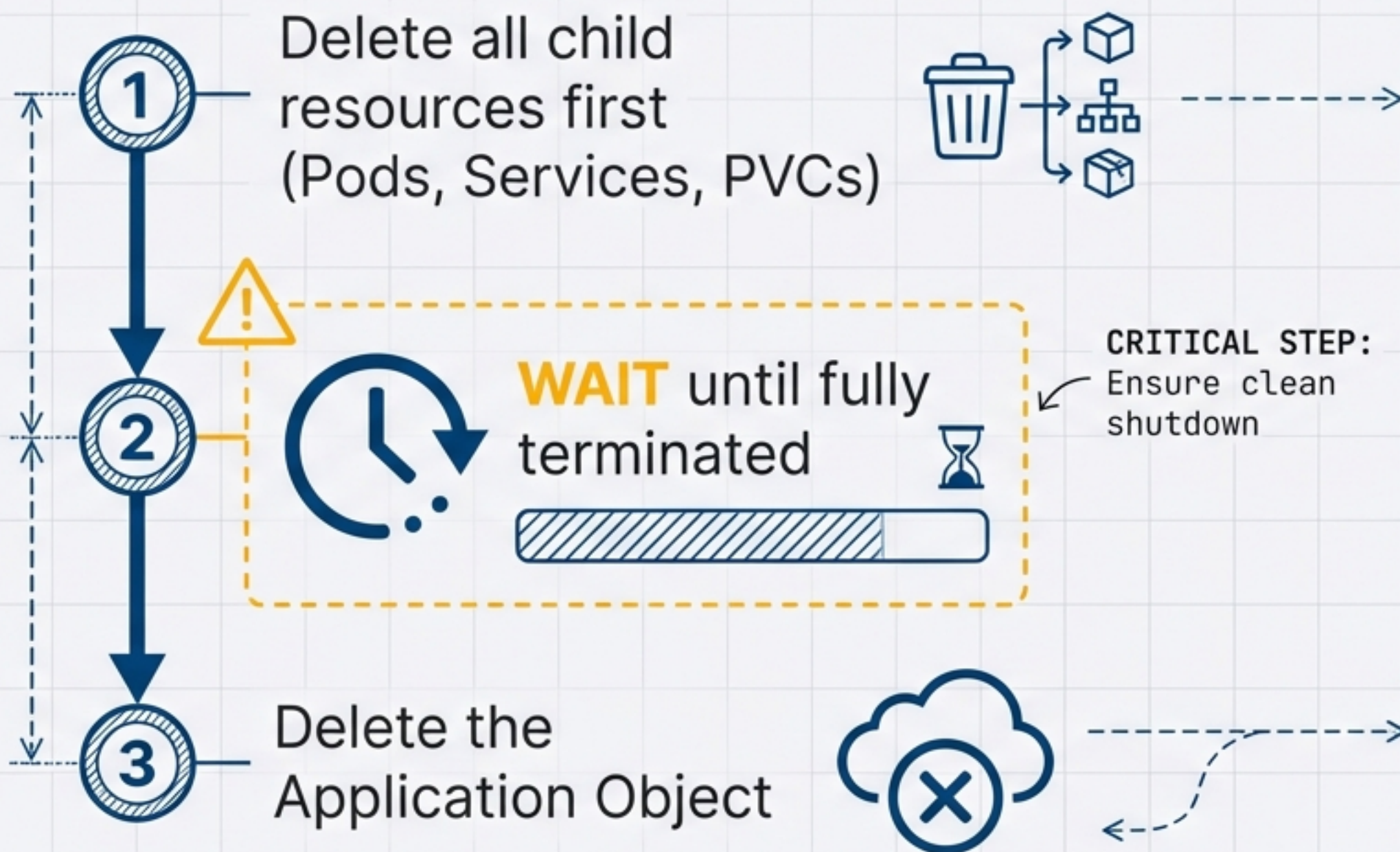
Lifecycle Management: Delete Propagation Policy



This is pure Kubernetes behavior surfaced through ArgoCD.

Strategy A: Foreground Deletion

**SAFETY
FIRST** ⚙️



When to use:

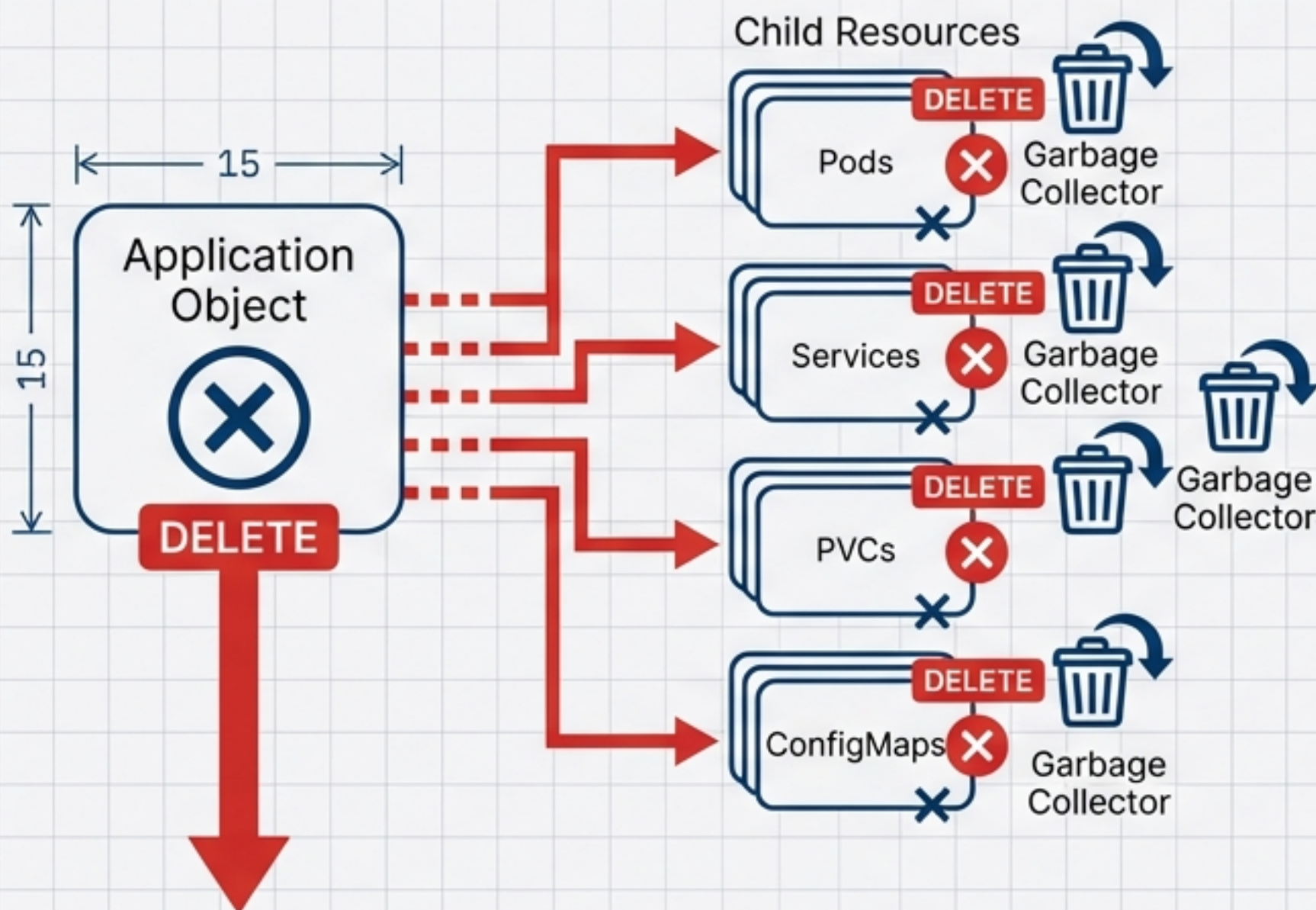
- Databases
- StatefulSets
- Services requiring clean shutdown
- Spring Boot Apps with persistence

* "Kubernetes Deletion Policy" in JetBrains Mono

Strategy B: Background Deletion



SPEED FIRST



Asynchronous / Parallel Deletion

k8s.io/docs/concepts/workloads/controllers/garbage-collection/

Use Cases:

When to use:

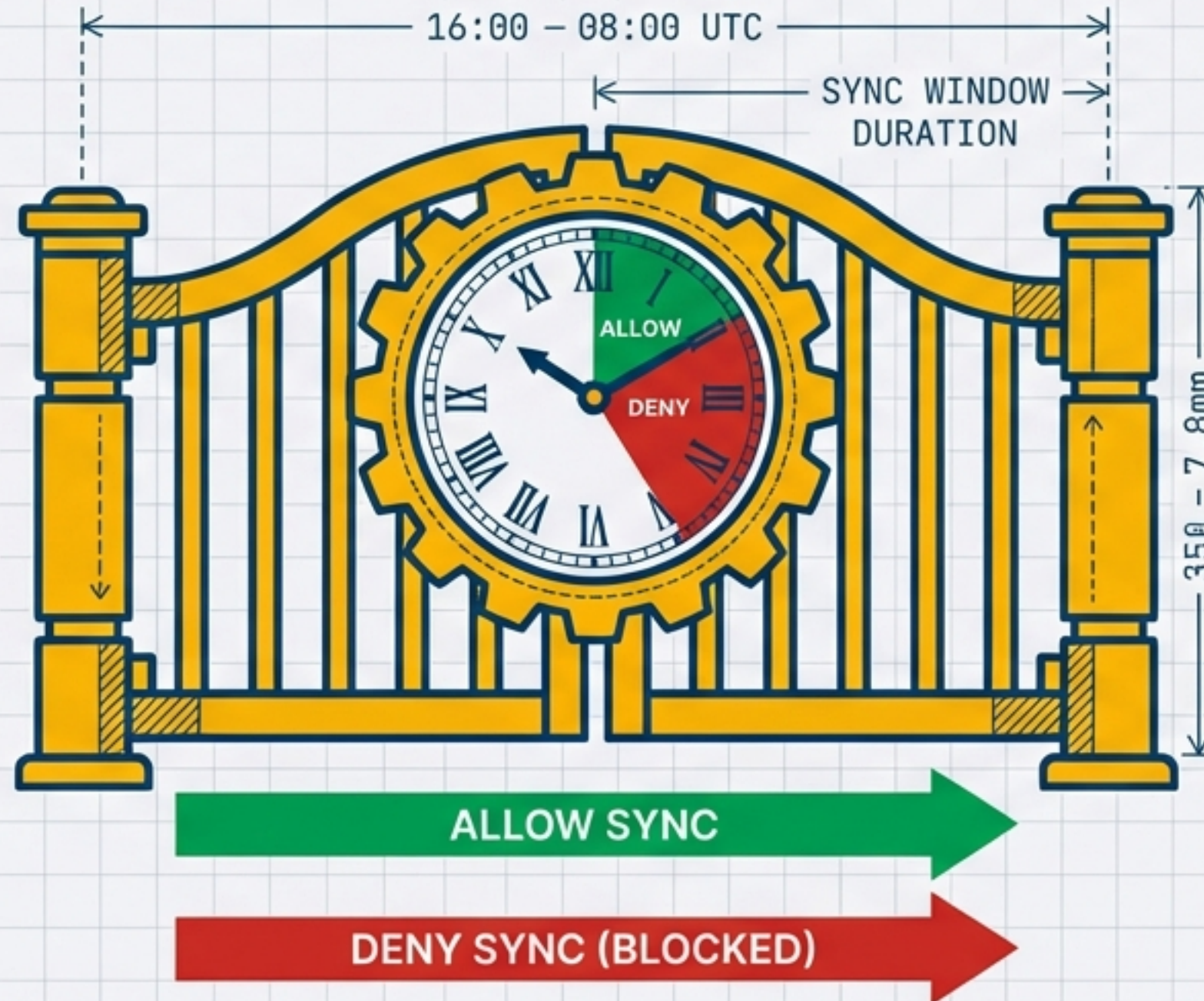
- Stateless Applications (React Frontends)
- Development Environments
- Ephemeral resources
- Scenarios requiring fast cleanup

Governance: Sync Windows

The Deployment Curfew



Time-based rules that tell ArgoCD to ALLOW or DENY syncs (auto or manual).

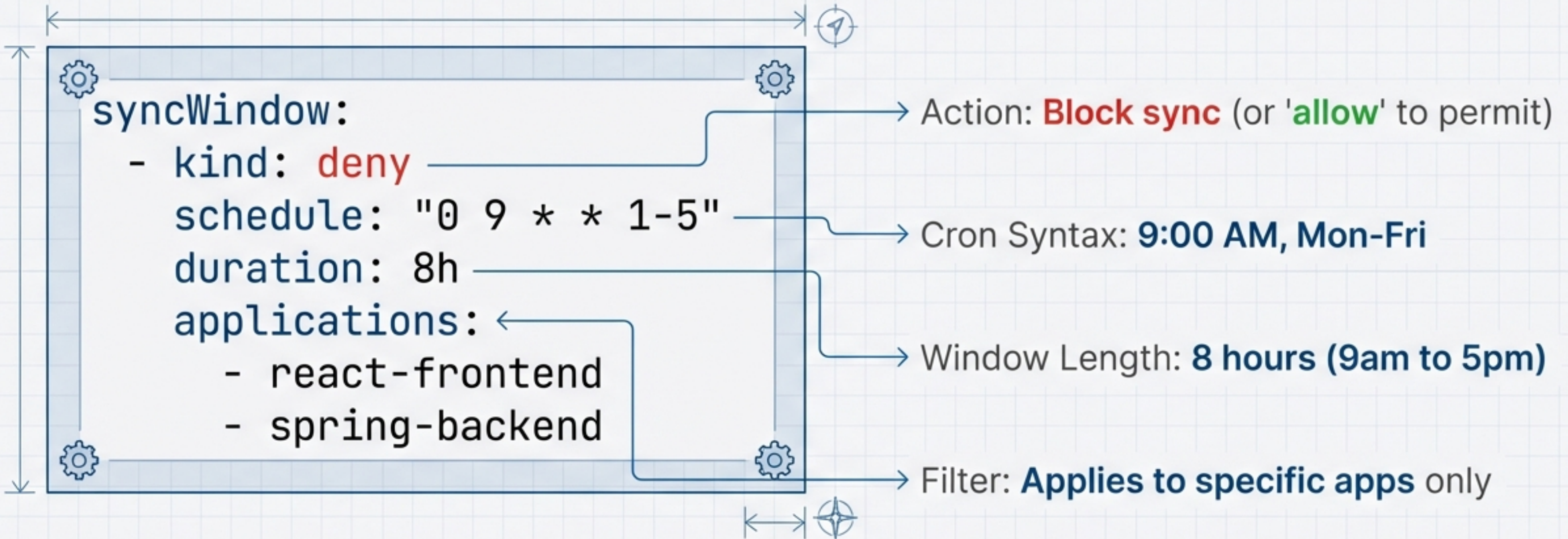


Why it exists

- Prevents deployments during business hours
- Blocks changes during incident windows
- Restricts auto-sync to night-time only

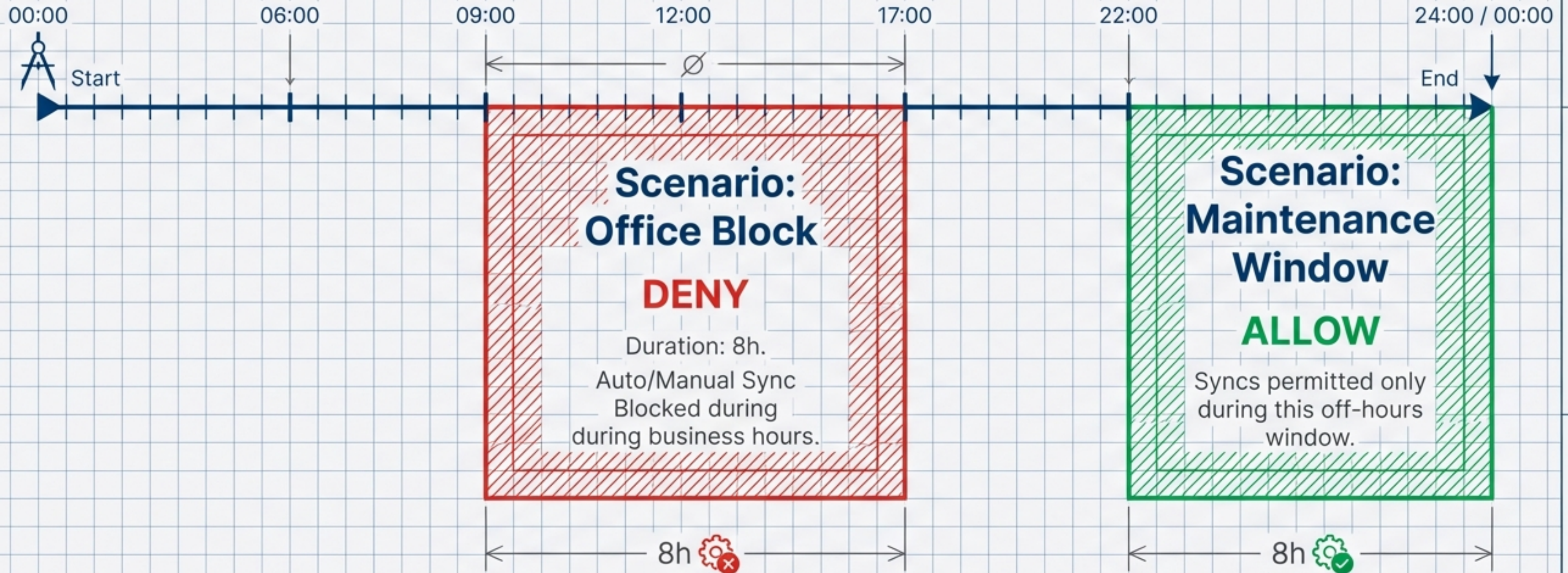
Configuring Time-Based Controls

Engineering Blue (#003366) Inter Tight



* 'ArgoCD Sync Windows Configuration' in JetBrains Mono

Sync Windows in Practice



* "Sync Windows in Practice" in Jet8rains Mono

The Operator's Cheat Sheet

Engineering Blue (#003366) Inter Tight

Self-Healing



- Corrects drift from "kubectl edit"
- Ensures Git consistency
- Logic: Detect → Reapply → Scale

Self-Prune

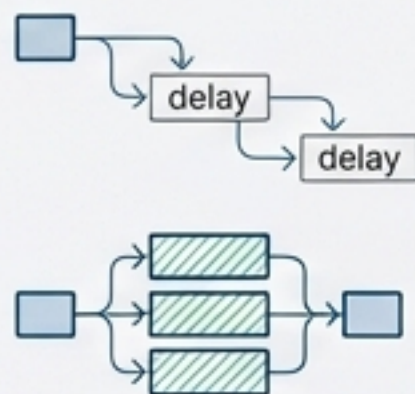


- Removes resources deleted from Git
- Prevents "Ghost Resources"
- **Without Prune** = Orphans / **With Prune** = Clean

Deletion Propagation



- **Foreground:** Wait for children. Safe/Slow (DBs)
- **Background:** Delete immediately. Fast/
Fast/Async (Stateless)



Sync Windows



- **Deny:** Block critical times (9-5 Office Hours) ❌
- **Allow:** Permit only during Maintenance (Nightly) ✅
- **Controls:** Schedule, Duration, App Filters ⚙️

